

Place and Route of the 8-bit Bit-Serial Adder

Taking the Lab 2 netlist through IC Compiler

Bibek Shrestha | October 2026 | Synopsys IC Compiler, SAED 90 nm

The assignment

Run the IC Compiler flow from the tutorial on a design from the previous lab: floorplan, power plan, placement, clock tree synthesis and routing. Generate and analyze the reports after CTS and after routing.

At a glance

	Lab 2 (DC)	After CTS	After routing
Library corner	typ	max	max
Wires	none	estimated	routed + extracted
Clock	5 ns	5 ns	5 ns
Setup slack	+1.77 ns	[ns]	[ns]
Violating paths	0	[]	[]
Hold violations		[]	[]
Flip-flops	23	[]	[]
Cell area	1268.12	[]	[]
Utilization		[%]	[%]
Open nets / DRCs			[] / []

Table 1: Lab 2 synthesis next to the two ICC checkpoints.

1 The design

The bit-serial adder from Lab 2 adds two 8-bit numbers with one full adder, one bit per clock. A 3-state FSM (`u_ctrl`) drives a datapath (`u_dp`) with two shift registers, a carry flip-flop and a 4-bit bit counter.

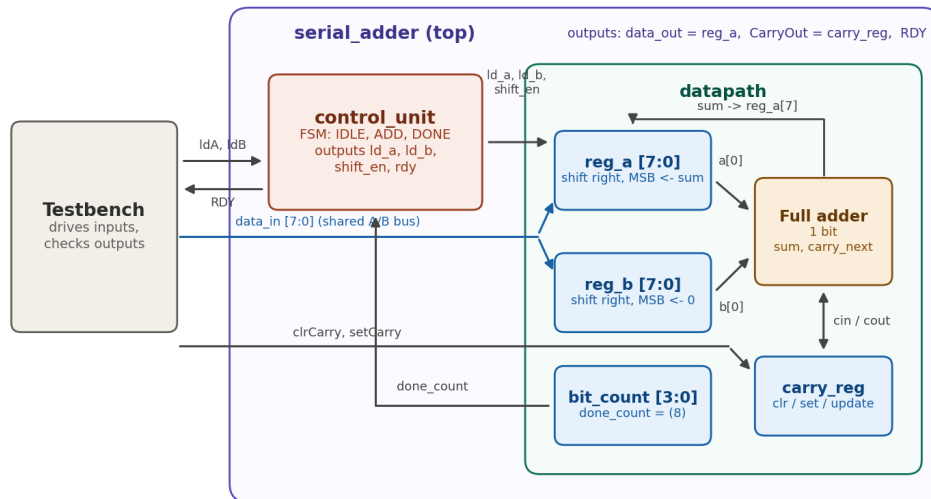


Figure 1: Block diagram from Lab 2.

Ports: `clk`, `rst_n`, `data_in[7:0]`, `data_out[7:0]`, `ldA`, `ldB`, `clrCarry`, `setCarry`, `CarryOut`, `RDY`: 24 signal pins.

2 Script

Same flow as the tutorial. Only the names change: library `ADDER_LIB`, design `adder_baseline.ddc` with top `serial_adder`, constraints `adder_baseline.sdc` (5 ns), and checkpoints `ADDER_init`, `ADDER_cts_opt`, `ADDER_route`. This design has no tri-state bus, so the BSLEX ban used for the multiplier is not needed here.

3 Routed layout

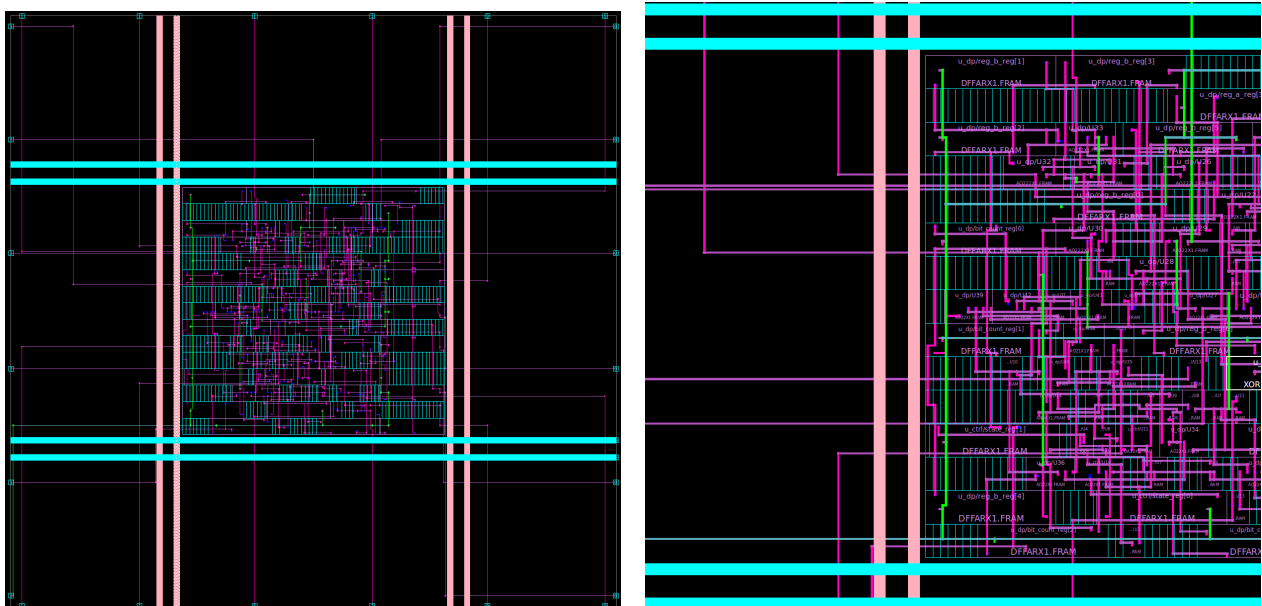


Figure 2: Left: the whole block. Right: the core, with fillers (cyan) between cells.

- **Ring size.** Querying the left VSS ring segment gives layer M6, width $1.0\mu\text{m}$ and length $103.2\mu\text{m}$, so the ring is about $103\mu\text{m}$ on a side.
- **Long I/O wires.** Most of the die is the $30\mu\text{m}$ margin. The 24 signal wires cross it from the pins on the die edge to the cells in the core, so for a design this small the I/O wiring is a large share of the total wire length.
- **All 23 flip-flops are visible**, the same count as in Lab 2:

```

u_dp/reg_a_reg[0..7]      8
u_dp/reg_b_reg[0..7]      8
u_dp/bit_count_reg[0..3]  4
u_dp/carry_reg_reg        1
u_ctrl/state_reg[0..1]    2

```

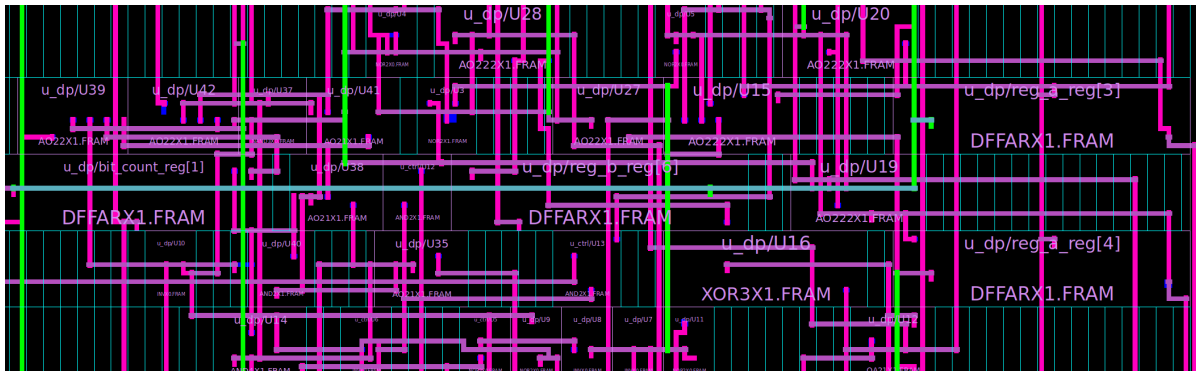


Figure 3: Closer view. `u_dp/U16` is an `XOR3X1`: the sum bit of the single full adder, $a \oplus b \oplus c$.

4 Reports

Report	What I checked	Result
<code>ADDER_route_qor.rpt</code>	slack, violating paths	<code>[]</code>
<code>ADDER_route_setup.rpt</code>	worst setup path	<code>[start → end, slack]</code>
<code>ADDER_route_hold.rpt</code>	worst hold path	<code>[]</code>
<code>ADDER_route_util.rpt</code>	standard cell utilization	<code>[]</code>
<code>verify_zrt_route</code>	open nets, DRCs	<code>[]</code>

Table 2: Post-route reports.

Compared with Lab 2

In Lab 2 the critical path was control logic (`1dB → FSM → bit_count_reg[3]`), not the adder, with `+1.77 ns` of slack at the typical corner. After routing at the slow corner the slack is `[ns]`, so the design `[still meets / misses]` 5 ns.

5 Inside the cells

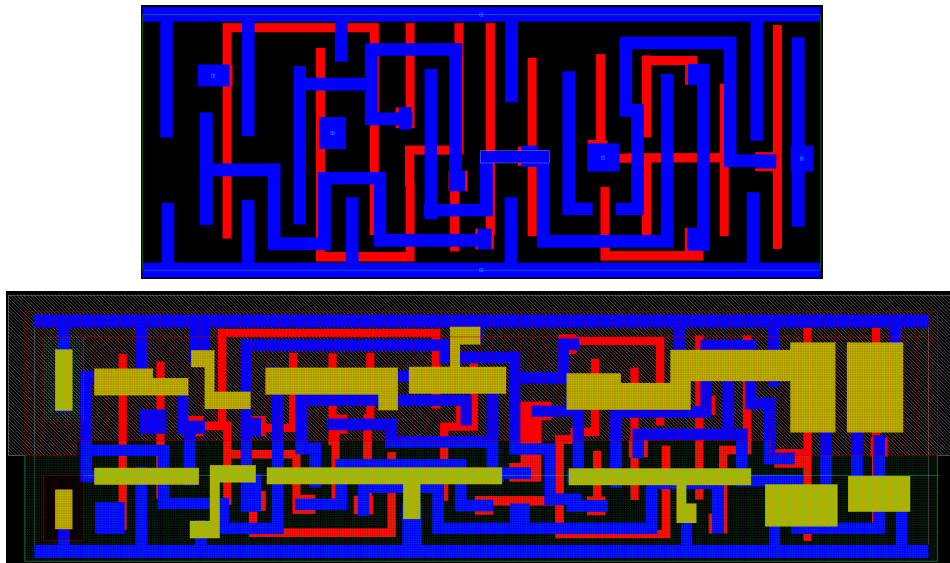


Figure 4: Top: XOR3X1 FRAM view (the adder's sum cell): rails, metal blockages and four pin squares (3 inputs, 1 output). Bottom: DFFARX1 CEL view, the flip-flop used for all 23 registers. It is much wider than a logic gate, which is why the flip-flops are the biggest cells in the layout.

6 What I learned

- In a small design the core is tiny; the fixed $30\mu\text{m}$ margin and the I/O wires take up most of the die.
- Every register from the RTL can be found by name in the layout, which makes checking the netlist easy.
- Flip-flops dominate the area here: 23 of them, each several times wider than an XOR.

A Source files

RTL: src/assignment2.sv

```
// =====
// Datapath: registers, full adder, carry register, bit counter
// =====

`timescale 1ns/10ps
module datapath (
    input logic      clk,
    input logic      rst_n,
    input logic [7:0] data_in,
    output logic [7:0] data_out,
    input logic      ld_a,
    input logic      ld_b,
    input logic      shift_en,
    input logic      clr_carry,
    input logic      set_carry,
    output logic      carry_out,
    output logic      done_count
);
```

```

logic [7:0] reg_a, reg_b;
logic      carry_reg;
logic [3:0] bit_count;
logic      sum, carry_next;

// Single 1-bit full adder (combinational)
assign sum      = reg_a[0] ^ reg_b[0] ^ carry_reg;
assign carry_next = (reg_a[0] & reg_b[0]) | (reg_a[0] & carry_reg) | (reg_b[0] & carry_reg);

always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        reg_a      <= 8'b0;
        reg_b      <= 8'b0;
        carry_reg <= 1'b0;
        bit_count <= 4'b0;
    end else begin
        if (ld_a)      reg_a <= data_in;
        else if (shift_en) reg_a <= {sum, reg_a[7:1]};

        if (ld_b)      reg_b <= data_in;
        else if (shift_en) reg_b <= {1'b0, reg_b[7:1]};

        if (clr_carry)  carry_reg <= 1'b0;
        else if (set_carry) carry_reg <= 1'b1;
        else if (shift_en) carry_reg <= carry_next;

        if (ld_b)      bit_count <= 4'b0;
        else if (shift_en) bit_count <= bit_count + 1'b1;
    end
end

assign data_out  = reg_a;
assign carry_out = carry_reg;
assign done_count = (bit_count == 4'd8);

endmodule

// =====
// Control unit: 3-state FSM (IDLE -> ADD -> DONE -> IDLE)
// =====
module control_unit (
    input logic clk,
    input logic rst_n,
    input logic ldA,
    input logic ldB,
    input logic done_count,
    output logic ld_a,
    output logic ld_b,
    output logic shift_en,
    output logic rdy
);

typedef enum logic [1:0] {IDLE, ADD, DONE} state_t;
state_t state, next_state;

always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) state <= IDLE;
    else      state <= next_state;
end

always_comb begin
    ld_a      = 1'b0;
    ld_b      = 1'b0;

```

```

    shift_en  = 1'b0;
    rdy       = 1'b0;
    next_state = state;

    case (state)
        IDLE: begin
            if (ldA) ld_a = 1'b1;
            if (ldB) begin
                ld_b      = 1'b1;
                next_state = ADD;
            end
        end

        ADD: begin
            if (!done_count) shift_en = 1'b1;
            if (done_count) next_state = DONE;
        end

        DONE: begin
            rdy      = 1'b1;
            next_state = IDLE;
        end

        default: next_state = IDLE;
    endcase
end

endmodule

// =====
// Top level: connects datapath + control unit, external interface
// =====
module serial_adder (
    input logic      clk,
    input logic      rst_n,
    input logic [7:0] data_in,
    output logic [7:0] data_out,
    input logic      ldA,
    input logic      ldB,
    input logic      clrCarry,
    input logic      setCarry,
    output logic      CarryOut,
    output logic      RDY
);

    logic ld_a, ld_b, shift_en, done_count;

    control_unit u_ctrl (
        .clk      (clk),
        .rst_n     (rst_n),
        .ldA      (ldA),
        .ldB      (ldB),
        .done_count (done_count),
        .ld_a     (ld_a),
        .ld_b     (ld_b),
        .shift_en  (shift_en),
        .rdy      (RDY)
    );

    datapath u_dp (
        .clk      (clk),
        .rst_n     (rst_n),
        .data_in   (data_in),
        .data_out  (data_out),

```

```

        .ld_a      (ld_a),
        .ld_b      (ld_b),
        .shift_en  (shift_en),
        .clr_carry  (clrCarry),
        .set_carry  (setCarry),
        .carry_out  (CarryOut),
        .done_count (done_count)
    );

endmodule

```

Testbench: src/assignment2_tb.sv

```

`timescale 1ns/10ps

module assignment2_tb;

    logic        clk = 0;
    logic        rst_n;
    logic [7:0]  data_in, data_out;
    logic        ldA, ldB, clrCarry, setCarry;
    logic        CarryOut, RDY;

    serial_adder dut (
        .clk      (clk),
        .rst_n    (rst_n),
        .data_in  (data_in),
        .data_out (data_out),
        .ldA      (ldA),
        .ldB      (ldB),
        .clrCarry (clrCarry),
        .setCarry (setCarry),
        .CarryOut (CarryOut),
        .RDY      (RDY)
    );

    always #5 clk = ~clk;    // 10ns clock period

    task automatic do_add(input [7:0] a, input [7:0] b);
        @(negedge clk);
        clrCarry = 1'b1;      // start each addition with carry cleared
        @(negedge clk);
        clrCarry = 1'b0;
        data_in = a; ldA = 1'b1;
        @(negedge clk);
        ldA = 1'b0;
        data_in = b; ldB = 1'b1;
        @(negedge clk);
        ldB = 1'b0;
        wait (RDY);
        @(negedge clk);
        $display("A=%0d B=%0d -> result=%0d carry=%0d", a, b, data_out, CarryOut);
    endtask

    task automatic do_add_cin(input [7:0] a, input [7:0] b);
        @(negedge clk);
        setCarry = 1'b1;      // carry-in = 1
        @(negedge clk);
        setCarry = 1'b0;
        data_in = a; ldA = 1'b1;
        @(negedge clk);
        ldA = 1'b0;

```

```

    data_in = b; ldB = 1'b1;
    @(negedge clk);
    ldB = 1'b0;
    wait (RDY);
    @(negedge clk);
    $display("A=%0d B=%0d +cin -> result=%0d carry=%0d", a, b, data_out, CarryOut);
endtask

initial begin
    rst_n = 0; ldA = 0; ldB = 0; clrCarry = 0; setCarry = 0; data_in = 0;
    @(negedge clk);
    @(negedge clk);
    rst_n = 1;

    do_add(8'd3, 8'd5); // expect result=8, carry=0
    do_add(8'd200, 8'd100); // expect result=44, carry=1 (300 wraps in 8 bits)
    do_add(8'd255, 8'd1); // expect result=0, carry=1
    do_add_cin(8'd3, 8'd5); // expect result=9, carry=0
    do_add_cin(8'd255, 8'd0); // expect result=0, carry=1

    #20 $finish;
end

endmodule

```

ICC script: src/icc_adder.tcl

```

# icc_adder.tcl : Lab 3, Assignment 2 (8-bit bit-serial adder, top = serial_adder)
# Run from icc_assignment2/, one level below output/ and constraints/:
# icc_shell -shared_license -f icc_adder.tcl | tee icc_adder.log

gui_start

set_app_var search_path /apps/designlib/SAED90_EDK/SAED_EDK90nm/Digital_Standard_cell_Library/
synopsys/models/
set_app_var target_library "saed90nm_max.db"
set_app_var link_library "* saed90nm_max.db"

create_mw_lib -technology "/apps/designlib/SAED90_EDK/SAED_EDK90nm/Technology_Kit/milkyway/
saed90nm_icc_1p9m.tf" -mw_reference_library "/apps/designlib/SAED90_EDK/SAED_EDK90nm/
Digital_Standard_cell_Library/process/astro/fram/saed90nm/" "ADDER_LIB"

open_mw_lib ADDER_LIB

import_designs "../output/adder_baseline.ddc" -format ddc -top "serial_adder" -cel "serial_adder"

read_sdc ../constraints/adder_baseline.sdc

set_tlu_plus_files -max_tluplus "/apps/designlib/SAED90_EDK/SAED_EDK90nm/Technology_Kit/starrcxt/
tluplus/saed90nm_1p9m_1t_Cmax.tluplus" -min_tluplus "/apps/designlib/SAED90_EDK/SAED_EDK90nm/
Technology_Kit/starrcxt/tluplus/saed90nm_1p9m_1t_Cmin.tluplus" -tech2itf_map "/apps/designlib/
SAED90_EDK/SAED_EDK90nm/Digital_Standard_cell_Library/process/astro/tech/tech2itf.map"

save_mw_cel -as ADDER_init

create_floorplan -core_utilization 0.6 -start_first_row -left_io2core "30" -bottom_io2core "30" -
right_io2core "30" -top_io2core "30"

derive_pg_connection -power_net "VDD" -power_pin "VDD" -ground_net "VSS" -ground_pin "VSS" -
create_ports "top"

```

```

create_rectangular_ring -nets {VSS} -left_offset 0.5 -left_segment_layer M6 -left_segment_width
    1.0 -extend_ll -extend_lh -right_offset 0.5 -right_segment_layer M6 -right_segment_width 1.0 -
    extend_rl -extend_rh -bottom_offset 0.5 -bottom_segment_layer M7 -bottom_segment_width 1.0 -
    extend_bl -extend_bh -top_offset 0.5 -top_segment_layer M7 -top_segment_width 1.0 -extend_tl -
    extend_th

create_rectangular_ring -nets {VDD} -left_offset 1.8 -left_segment_layer M6 -left_segment_width
    1.0 -extend_ll -extend_lh -right_offset 1.8 -right_segment_layer M6 -right_segment_width 1.0 -
    extend_rl -extend_rh -bottom_offset 1.8 -bottom_segment_layer M7 -bottom_segment_width 1.0 -
    extend_bl -extend_bh -top_offset 1.8 -top_segment_layer M7 -top_segment_width 1.0 -extend_tl -
    extend_th

create_power_strap -nets {VSS} -layer M6 -direction vertical -width 3
create_power_strap -nets {VDD} -layer M6 -direction vertical -width 3

create_fp_placement

place_opt
clock_opt

save_mw_cel -as ADDER_cts_opt

report_placement_utilization > ../output/ADDER_cts_util.rpt
report_qor > ../output/ADDER_cts_qor.rpt
report_timing -delay max -max_paths 5 > ../output/ADDER_cts.setup.rpt
report_timing -delay min -max_paths 5 > ../output/ADDER_cts.hold.rpt

insert_stdcell_filler -cell_with_metal "SHFILL2" -cell_without_metal "SHFILL2" -connect_to_power {
    VDD} -connect_to_ground {VSS}

route_opt
verify_zrt_route

report_placement_utilization > ../output/ADDER_route_util.rpt
report_qor > ../output/ADDER_route_qor.rpt
report_timing -delay max -max_paths 5 > ../output/ADDER_route.setup.rpt
report_timing -delay min -max_paths 5 > ../output/ADDER_route.hold.rpt

extract_rc -coupling_cap
write_parasitics -format SBPF -output ../output/ADDER.output.sbpf
write_verilog ../output/ADDER_LIB.output.v

save_mw_cel -as ADDER_route

```